

TRABAJO DE FIN DE GRADO

Orchestration Trade-offs in Edge Computing: Systemd vs Kubernetes

Análisis de rendimiento, distribución de cargas e inferencia de modelos de IA en el extremo de la red

Autor:

[Tu Nombre y Apellidos]

Director/es:

[Nombre del Director/a]

Grado en Ingeniería [Informática / Telecomunicaciones]

[Nombre de tu Universidad]

[Mes y Año de la convocatoria]

Índice

1	Introducción	1
1.1	Contexto y Motivación	1
1.2	Objetivos del Trabajo	1
1.3	Estructura de la Memoria	2
2	Estado del Arte y Tecnologías Involucradas	3
2.1	Edge Computing y Orquestación	3
2.2	Gestión de Contenedores	3
2.3	Protocolos de Comunicación y Serialización	3
2.4	Machine Learning Distribuido	4
3	Diseño del Sistema y Entorno de Pruebas	5
3.1	Infraestructura Física y Virtual	5
3.2	Aprovisionamiento y Despliegue Agnóstico	5
3.3	Arquitectura Lógica de la Solución	5
4	Fase 1: Transmisión de Datos en Systemd (Conteo de Imágenes)	7
4.1	Definición de la Tarea	7
4.2	Diseño del Experimento (Comparativa)	7
4.3	Resultados y Análisis de Rendimiento	7
4.4	Conclusiones de la Fase 1	8
5	Fase 2: Despliegue e Inferencia de Modelos de IA	9
5.1	Entrenamiento del Modelo en el Master	9
5.2	Distribución del Modelo y Datos	9
5.3	Ejecución de Inferencia en el Edge (Workers)	9
5.4	Métricas de Procesamiento (T_{proc})	9
6	Fase 3: Migración a Kubernetes (K3s) y Análisis de Trade-offs	10
6.1	Migración de la Infraestructura a K3s	10
6.2	Ejecución de Cargas de Trabajo (Fase 1 y 2 en K8s)	10
6.3	Comparativa Directa: Systemd vs Kubernetes	10
7	Conclusiones y Trabajo Futuro	11
7.1	Conclusiones Finales	11
7.2	Trabajo Futuro	11

Índice de figuras

3.1 Topología del sistema y flujos de comunicación.	6
---	---

Índice de cuadros

4.1	Resumen de resultados de transmisión para 60,000 imágenes.	8
-----	--	---

Capítulo 1

Introducción

Este capítulo debe introducir al lector al problema, por qué es importante y qué vas a resolver.

1.1 Contexto y Motivación

Aquí debes hablar sobre el auge del *Edge Computing* (computación en el extremo) y cómo difiere del *Cloud Computing* tradicional. Explica que los entornos Edge tienen recursos limitados (CPU, RAM, red inestable) y que las herramientas de orquestación diseñadas para la nube (como Kubernetes) pueden ser demasiado pesadas. Introduce el dilema: ¿Necesitamos orquestadores complejos o bastan herramientas nativas del sistema operativo como Systemd + Podman?

1.2 Objetivos del Trabajo

- **Objetivo Principal:** Analizar y comparar los *trade-offs* (rendimiento, resiliencia y complejidad) entre Kubernetes y Systemd como orquestadores para cargas de trabajo Edge.
- **Objetivos Específicos:**
 - Desplegar una infraestructura Master-Worker utilizando Ansible, Podman y Systemd (Fase 1).
 - Evaluar el rendimiento de múltiples protocolos de red (HTTP, gRPC, ZeroMQ, MQTT) y formatos de serialización (JSON, Protobuf, MessagePack) para la transmisión masiva de datos (Dataset MNIST).
 - Implementar un flujo de trabajo de Machine Learning distribuido (Fase 2) entrenando un modelo PyTorch en el nodo máster e infiriendo en los nodos worker.
 - Migrar la arquitectura a un entorno de Kubernetes/K3s (Fase 3) y contrastar las métricas de consumo y latencia frente a Systemd.

1.3 Estructura de la Memoria

Breve resumen de lo que contiene cada uno de los siguientes capítulos.

Capítulo 2

Estado del Arte y Tecnologías Involucradas

Define teóricamente las herramientas y conceptos que usas. No es un manual de uso, sino una justificación técnica.

2.1 Edge Computing y Orquestación

Diferencias entre Cloud, Fog y Edge. Desafíos en el Edge (huella de memoria, latencia, ancho de banda).

2.2 Gestión de Contenedores

- **Podman:** Arquitectura *daemonless* y *rootless*. Por qué es mejor que Docker para el Edge.
- **Systemd Quadlets:** Cómo Systemd integra de forma nativa la gestión del ciclo de vida de los contenedores.
- **Kubernetes (K3s):** Distribuciones ligeras de Kubernetes para IoT/Edge. Arquitectura del plano de control y el CNI (Container Network Interface).

2.3 Protocolos de Comunicación y Serialización

Explica la teoría y diferencias de los que has comparado:

- **REST / HTTP(s) y JSON:** Estándar web, basado en texto. Cuellos de botella en el parseo.
- **gRPC y Protocol Buffers:** Llamadas a procedimientos remotos sobre HTTP/2. Serialización binaria estructurada y fuertemente tipada.
- **ZeroMQ:** Sockets TCP crudos sin *broker* (Brokerless P2P). Alta velocidad.

- **MQTT:** Patrón Pub/Sub con *broker* intermediario.
- **MessagePack:** Serialización binaria sin esquema (*schema-less*), flexible y rápida.

2.4 Machine Learning Distribuido

Breve introducción a PyTorch, inferencia de modelos, y el dataset MNIST.

Capítulo 3

Diseño del Sistema y Entorno de Pruebas

3.1 Infraestructura Física y Virtual

Detalla el hardware:

- Nodo Master (Padre): Ubuntu 22.04.5 bajo WSL en Windows 11. Rol: Orquestador (Ansible), entrenamiento de IA, y despachador de tareas.
- Nodos Workers (Hijos): 2 Máquinas Virtuales Ubuntu 24.04.4 en VMware. Rol: Inferencia, cálculo, retorno de métricas.

3.2 Aprovisionamiento y Despliegue Agnóstico

Explica la separación estricta entre *Build* (construcción en el máster) y *Run* (ejecución en el worker). Detalla cómo Ansible transporta los artefactos locales (‘.tar’) usando ‘podman load’ (estrategia *side-loading*) para ahorrar ancho de banda y CPU, evitando repositorios externos (Docker Hub).

3.3 Arquitectura Lógica de la Solución

Aquí debes poner el esquema de la arquitectura Master-Worker, mostrando cómo fluyen los datos (Fase 2, Fase 3, Fase 4).

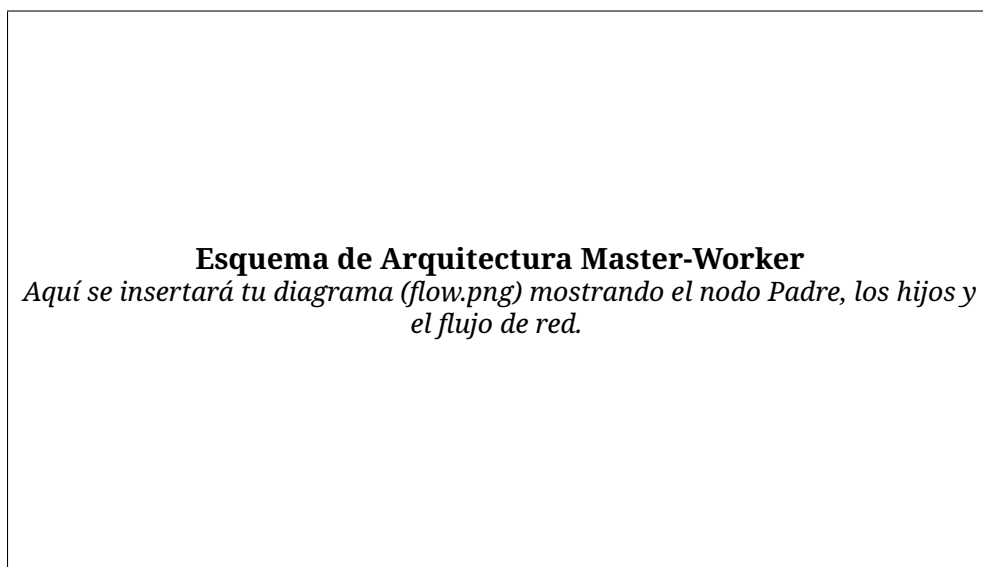


Figura 3.1: Topología del sistema y flujos de comunicación.

Capítulo 4

Fase 1: Transmisión de Datos en Systemd (Conteo de Imágenes)

En este capítulo desarrollas todo lo que me has pasado sobre el conteo de imágenes y la comparativa de red.

4.1 Definición de la Tarea

El Master descarga el dataset MNIST (60,000 imágenes), lo carga en memoria, lo divide en N particiones ($tamaño_particion = total_imagenes / N_nodos$) y envía los lotes. El Worker cuenta las imágenes recibidas y devuelve el total al Master.

4.2 Diseño del Experimento (Comparativa)

Explica la justificación de cada prueba:

- **Línea Base:** HTTP/1.1 + JSON (Demostrar ineficiencia del texto plano).
- **Estándar Microservicios:** gRPC + Protobuf (HTTP/2 binario).
- **Rendimiento Crudo:** ZeroMQ + Protobuf (Límite físico P2P).
- **IoT Broker:** MQTT + Protobuf (Penalización por doble salto).
- **Flexibilidad:** ZeroMQ + MessagePack (Esquema estático vs dinámico).

4.3 Resultados y Análisis de Rendimiento

Inserta aquí las tablas y gráficas de la Fase 1. Comenta el consumo extremo de RAM del JSON frente al paradigma binario (*Zero-Copy* con `np.frombuffer`).

Protocolo	Formato	Tiempo RTT (ms)	Throughput (img/s)	RAM Pico (MB)
HTTP/REST	JSON	≈ 16500	3,636	2600
gRPC	Protobuf	≈ 1037	57,850	384
ZeroMQ	Protobuf	≈ 672	89,285	250
ZeroMQ	MsgPack	≈ 540	111,087	209

Cuadro 4.1: Resumen de resultados de transmisión para 60,000 imágenes.

4.4 Conclusiones de la Fase 1

Explica cómo "La tríada ZeroMQ + MessagePack + Zero-Copy se establece como la arquitectura ganadora" dentro de Systemd.

Capítulo 5

Fase 2: Despliegue e Inferencia de Modelos de IA

Este capítulo evoluciona el envío tonto de imágenes a una tarea computacional real.

5.1 Entrenamiento del Modelo en el Master

Explica cómo el nodo Master utiliza PyTorch para entrenar un modelo clasificador convolucional (CNN) sencillo sobre el dataset MNIST. Detalla las métricas del entrenamiento (Epochs, Loss, Accuracy final).

5.2 Distribución del Modelo y Datos

Explica cómo se envía el modelo entrenado (ej. serializado con 'torch.save' en formato '.pt' o '.pth') hacia los workers utilizando la arquitectura de red ganadora de la Fase 1 (ZeroMQ). Seguidamente, se envían las particiones de imágenes.

5.3 Ejecución de Inferencia en el Edge (Workers)

Los nodos hijos reciben el modelo, lo cargan en memoria ('torch.load'), y para cada imagen recibida realizan una inferencia (predicción del número escrito a mano).

5.4 Métricas de Procesamiento (T_{proc})

- Tiempo medio de inferencia por lote.
- Consumo de CPU de los workers durante la carga computacional (Tensores IA).
- Fiabilidad: Exactitud de los números predichos y devueltos al máster.

Capítulo 6

Fase 3: Migración a Kubernetes (K3s) y Análisis de Trade-offs

Aquí culmina el propósito del TFG: la comparación directa de orquestadores.

6.1 Migración de la Infraestructura a K3s

Explica la configuración del cluster Kubernetes.

- Master asume el rol de Control Plane.
- Workers asumen roles de Kubelets.
- Traducción de los servicios ‘Systemd Quadlets’ a manifiestos de Kubernetes (‘Deployments’, ‘Services’, ‘Pods’).

6.2 Ejecución de Cargas de Trabajo (Fase 1 y 2 en K8s)

Se repite el envío masivo de imágenes y la inferencia de IA, pero ahora a través de la red virtual de Kubernetes (ej. Flannel o Calico CNI) y Kube-proxy.

6.3 Comparativa Directa: Systemd vs Kubernetes

Aquí comparas los resultados de Systemd frente a K8s usando las métricas definidas:

- **Plano de Gestión (Reposo):** Consumo de RAM y CPU de Systemd+Podman inactivo frente al Kubelet+Kube-proxy+Etcd inactivos en el Edge.
- **Plano de Datos (Red):** Cuánta latencia extra añade el CNI de K8s a gRPC y ZeroMQ frente a la comunicación nativa P2P de Podman.
- **Complejidad Operativa:** Facilidad de despliegue usando Ansible estático vs. mantenimiento del estado en K8s.

Capítulo 7

Conclusiones y Trabajo Futuro

7.1 Conclusiones Finales

Resumen general. Responde a la pregunta del TFG: ¿Vale la pena el peso de Kubernetes en el Edge, o es Systemd una alternativa viable? Destaca los números clave: 1) La inviabilidad de REST/JSON. 2) La eficiencia extrema de ZeroMQ+MsgPack en Podman. 3) El *overhead* demostrado por Kubernetes frente al despliegue desnudo.

7.2 Trabajo Futuro

Sugerencias para ampliar el trabajo:

- Probar aceleradores hardware en los Workers (GPU/NPU).
- Usar un framework de IA específico para Edge como TensorFlow Lite o ONNX Runtime.
- Implementar mecanismos de tolerancia a fallos automáticos en el lado de Systemd.

Bibliografía

- [1] Red Hat, *Podman Documentation*, [Online]. Disponible en: <https://podman.io/docs/>
- [2] Freedesktop.org, *systemd.service — Service unit configuration*, [Online].
- [3] Cloud Native Computing Foundation (CNCF), *gRPC: A high performance, open source universal RPC framework*, [Online].
- [4] P. Hintjens, *ZeroMQ: Messaging for Many Applications*, O'Reilly Media, 2013.
- [5] Rancher Labs (SUSE), *K3s Lightweight Kubernetes*, [Online]. Disponible en: <https://k3s.io/>
- [6] Paszke, A. et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, NeurIPS 2019.